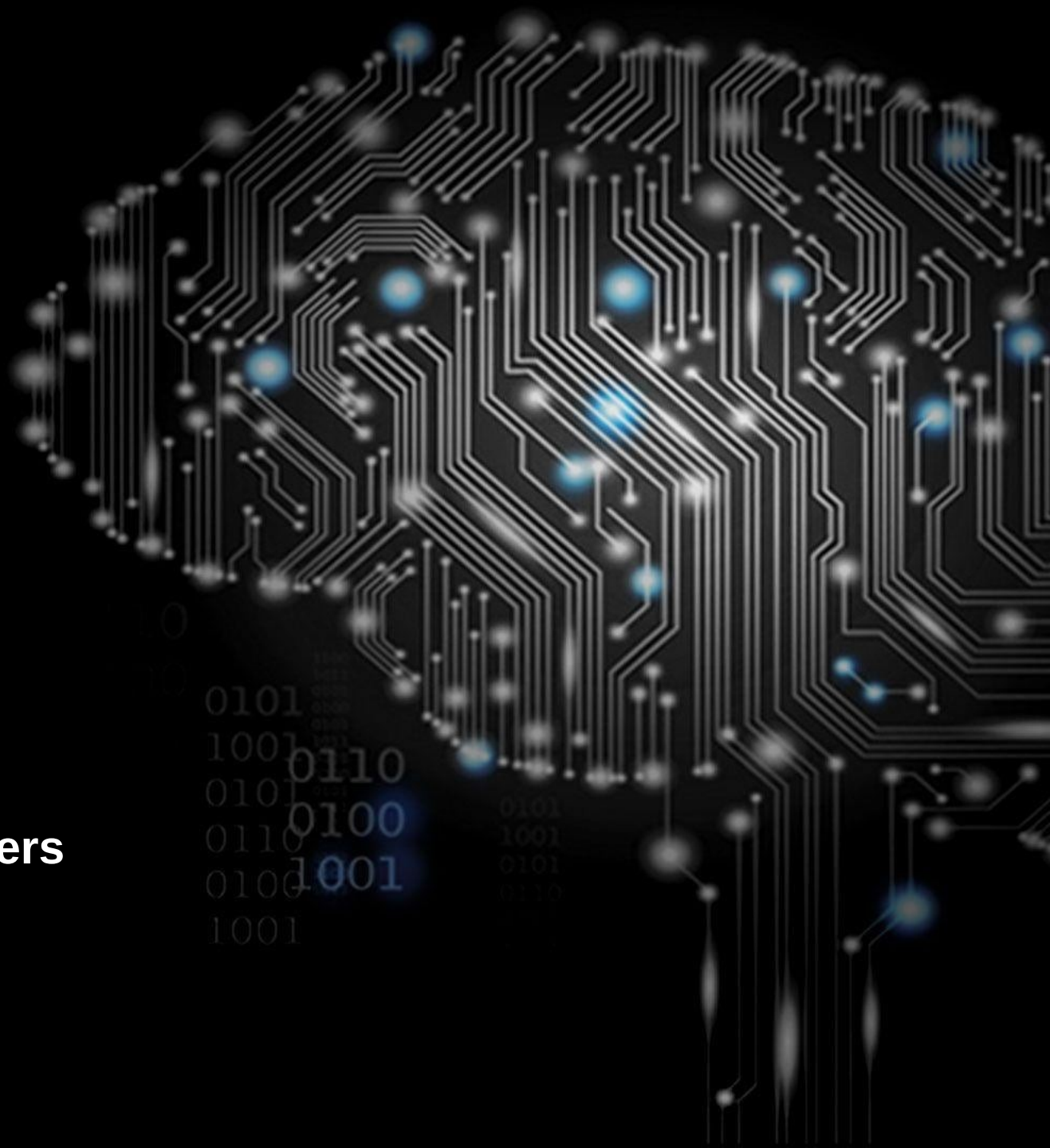


Introduction to Generative AI for Developers

6. Observability and QA

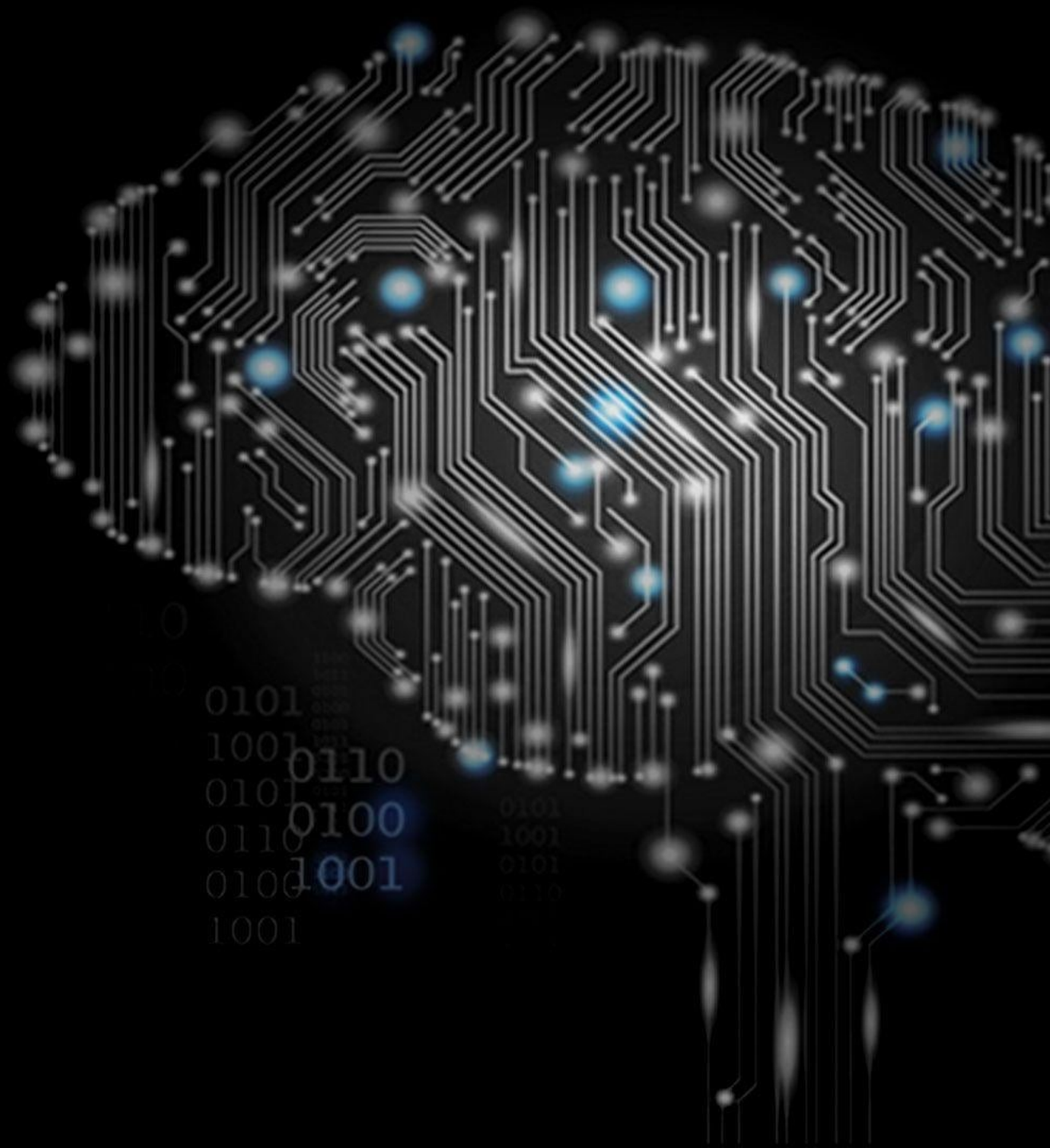
Sections

- What to observe
- Spring AI instrumentation and OpenTelemetry
- Traces, metrics, dashboards and alerts
- Logging prompts and responses safely
- QA in production



Observability and QA

1, What to observe



Why LLM features need more than standard monitoring

- An HTTP 200 response says nothing about whether the answer was correct, on-brand or compliant
- Every request has a direct, variable cost, which can grow quickly with traffic, retries or agent loops
- Behaviour can change without a deployment
  - Provider model updates, changes in customer language, new or updated documents in the vector store
  - Canary rollouts of prompts and models
- Failures are often silent
  - Refusals, empty answers after hitting an output limit, validation retries, guardrail blocks
- A single request can involve many steps: guardrails, retrieval, model calls, tool calls, retries
  - Tracing is needed to see where time, tokens and failures occu

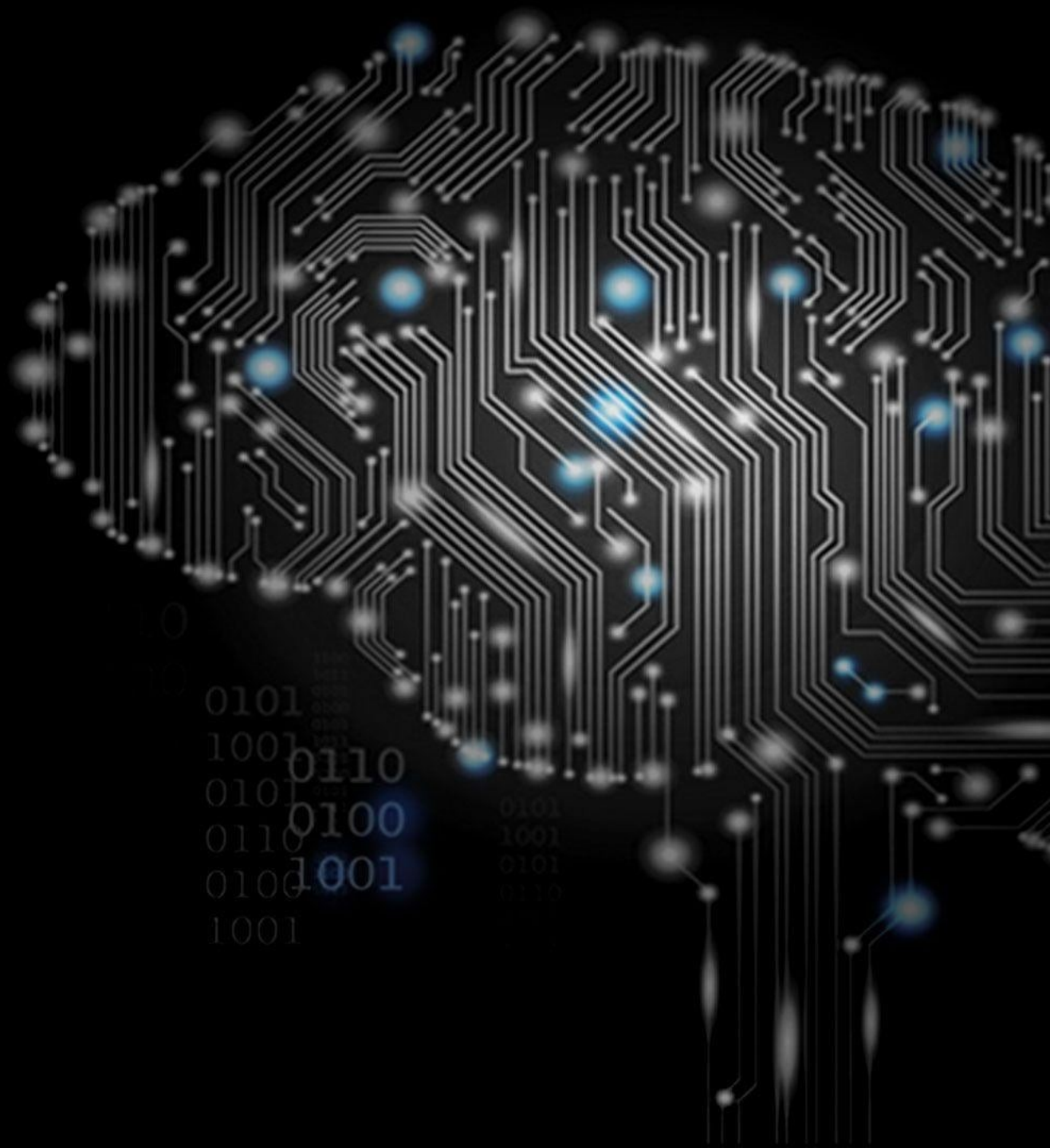
Signals to observe

- Operational
  - Latency (end-to-end and per step), error rates by type, timeouts, rate-limit responses, retries
- Cost
  - Input, output, cached and reasoning tokens by model and feature; cost per request and per day
- Quality
  - Refusal rate, schema validation failures and self-correction retries, incomplete responses, grounding and citation failures, online evaluation scores, user feedback
- Safety and compliance
  - Guardrail triggers by guardrail and decision, PII detections, injection detections, escalations to humans
- Agents and tools
  - Tool calls by tool, tool errors, iterations per request, human approvals requested and granted
- Retrieval
  - Documents retrieved per request, similarity scores, empty retrievals, retrieval latency



Service level objectives for an LLM feature

- Define objectives with the business before go-live, as for any service
- Examples for the triage service
  - 95 percent of triage requests complete within 4 seconds
  - At least 99 percent of responses pass schema validation without falling back to a human
  - Cost per 1,000 messages stays below an agreed budget
  - Refusal rate stays within an agreed band, for example 2 to 8 percent
  - Online evaluation pass rate stays within 3 points of the release baseline
- Objectives drive alerts and release decisions; without them, a dashboard has no definition of healthy



Observability and QA

2. Spring AI instrumentation and OpenTelemetry

The Micrometer Observation API

- Spring's observability is built on Micrometer's Observation API
  - One instrumentation point produces metrics, traces and log correlation
- Spring Boot exports metrics to systems such as Prometheus and traces through OpenTelemetry
- Spring AI instruments its components with observations, so most of the signals discussed earlier ppear without code changes
- Applications add their own observations for business-level detail



What Spring AI instruments

- ChatClient (spring.ai.chat.client): each call or stream, with the advisors and tools involved
- Advisors (spring.ai.advisor): each advisor's execution time, including your guardrail advisors
- Chat model (gen_ai.client.operation): each provider call, with model, token usage and finish reasons
- Tool calls (spring.ai.tool): each tool execution, with the tool name
- Embedding models (gen_ai.client.operation): embedding calls and their token usage
- Vector stores (db.vector.client.operation): add, delete and query operations, with top-k and similarity threshold
- Token usage metric: gen_ai.client.token.usage, by token type (input, output, total), provider and model

```
# As exported to Prometheus
gen_ai_client_operation_seconds_count{gen_ai_request_model="gpt-6-luna",...}
gen_ai_client_token_usage_total{gen_ai_token_type="output",gen_ai_request_model="gpt-6-luna",...}
gen_ai_chat_client_operation_seconds_sum{spring_ai_kind="chat_client",...}
db_vector_client_operation_seconds_count{db_operation_name="query",...}
```



Low- and high-cardinality data

- Low-cardinality keys have a small, fixed set of values; they become metric tags and trace attributes
  - Operation name, provider, requested and response model, advisor name, tool name, vector store operation
- High-cardinality keys have many distinct values; they appear only on traces
  - Conversation ID, request parameters, response ID, finish reasons, per-request token counts, tool call IDs, query filters
- Why it matters
  - High-cardinality metric tags create a separate time series per value and can overload the metrics system
  - Customer IDs, prompt text and conversation IDs must never be metric tags
- Apply the same rule to custom observations: prompt version and category are low cardinality; customer ID is not



OpenTelemetry GenAI semantic conventions

- OpenTelemetry defines standard names for GenAI spans, metrics and events, so tools from different vendors can interpret the same data
  - Core metrics: gen_ai.client.operation.duration and gen_ai.client.token.usage
  - Core attributes: operation name, provider, request and response model, input and output token counts
  - Agent and tool spans, such as invoke_agent and execute_tool, and conventions for the Model Context Protocol
- Status: the GenAI conventions were still in Development status as of mid-2026 and continue to change
  - In version 1.37, gen_ai.system was replaced by gen_ai.provider.name
  - Instrumentations may emit older names unless opted in to the latest conventions
- Content capture (prompts and completions) is off by default in the conventions, with options to omit it, attach it to spans, or store it externally with a reference
- Spring AI uses gen_ai.* names based on the conventions, with some differences
  - For example, it records gen_ai.system, and adds spring.ai.* and db.vector.* keys of its own



Adding business-level observations

- Built-in observations describe the mechanics; add observations for what the business cares about
  - Prompt version, category assigned, guardrail outcome, escalation to a human
- Use the Observation API so the data appears in both traces and metrics
- Use counters for decisions such as guardrail outcomes

```
TriageResult triage(String message, String channel) {
    PromptRef ref = prompts.ref("triage");
    Observation obs = Observation.createNotStarted("triage.request", observationRegistry)
            .lowCardinalityKeyValue("prompt.version", ref.version())
            .lowCardinalityKeyValue("channel", channel)
            .start();
    try (Observation.Scope scope = obs.openScope()) {
        TriageResult result = callModel(ref, message, channel);
        obs.lowCardinalityKeyValue("category", result.category().name());
        obs.lowCardinalityKeyValue("urgency", result.urgency().name());
        return result;
    } catch (RuntimeException e) {
        obs.error(e);
        throw e;
    } finally {
        obs.stop();
    }
}

// In each guardrail advisor
meterRegistry.counter("guardrail.decisions",
        "guardrail", "denied-topic", "decision", "block").increment();
```



Adding business-level observations

- Built-in observations describe the mechanics; add observations for what the business cares about
  - Prompt version, category assigned, guardrail outcome, escalation to a human
- Use the Observation API so the data appears in both traces and metrics
- Use counters for decisions such as guardrail outcomes

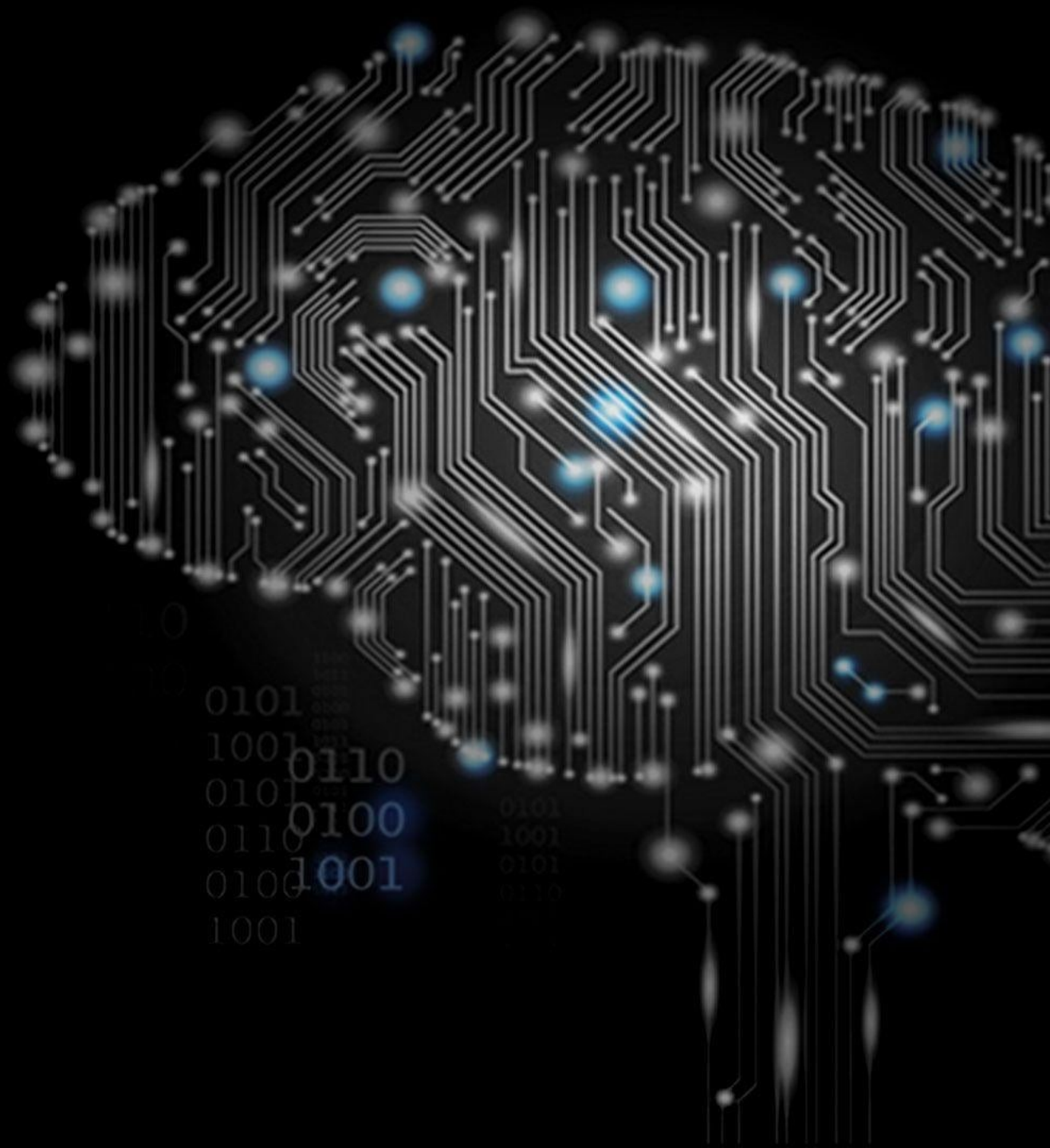
```
TriageResult triage(String message, String channel) {
    PromptRef ref = prompts.ref("triage");
    Observation obs = Observation.createNotStarted("triage.request", observationRegistry)
            .lowCardinalityKeyValue("prompt.version", ref.version())
            .lowCardinalityKeyValue("channel", channel)
            .start();
    try (Observation.Scope scope = obs.openScope()) {
        TriageResult result = callModel(ref, message, channel);
        obs.lowCardinalityKeyValue("category", result.category().name());
        obs.lowCardinalityKeyValue("urgency", result.urgency().name());
        return result;
    } catch (RuntimeException e) {
        obs.error(e);
        throw e;
    } finally {
        obs.stop();
    }
}

// In each guardrail advisor
meterRegistry.counter("guardrail.decisions",
        "guardrail", "denied-topic", "decision", "block").increment();
```



Observability and QA

3 Traces, metrics, dashboards and alerts



Exporting traces and metrics from Spring Boot 4

- Traces: Spring Boot 4 provides spring-boot-starter-opentelemetry for OTLP export through Micrometer Tracing
- Metrics: micrometer-registry-prometheus plus an exposed /actuator/prometheus endpoint
- Sampling: Spring Boot samples 10 percent of traces by default; development and labs use 100 percent
- Enable histograms for model latency so percentiles can be calculated in Prometheus

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-opentelemetry</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health, prometheus
  tracing:
    sampling:
      probability: 1.0          # labs only; production uses a
  opentelemetry:
    tracing:
      export:
        otlp:
          endpoint: http://localhost:4318/v1/traces
  metrics:
    distribution:
      percentiles-histogram:
        gen_ai.client.operation: true
```

Reading the trace of one request

- A trace shows every step of a request as nested spans with timing and attributes
- Typical shape for one triage request with guardrails and retrieval

```
POST /triage                                                  3.42 s
 └─ triage.request  prompt.version=v2 category=CARD           3.38 s
    └─ spring.ai.chat.client                                  3.36 s
       ├─ spring.ai.advisor  pii-masking                       0.01 s
       ├─ spring.ai.advisor  denied-topic                      0.62 s
       │   └─ gen_ai.client.operation  gpt-6-luna              0.60 s
       ├─ spring.ai.advisor  QuestionAnswerAdvisor             0.18 s
       │   └─ db.vector.client.operation  query top_k=5        0.15 s
       ├─ gen_ai.client.operation  gpt-6-luna                  2.31 s   in=1,840 out=260
       └─ spring.ai.advisor  compliance-output                 0.21 s
```

Useful Prometheus queries

- Latency, token usage, cost, guardrails and errors can all be expressed with a few queries

```
# p95 model call latency by model (requires histogram buckets)
histogram_quantile(0.95,
  sum by (le, gen_ai_request_model) (rate(gen_ai_client_operation_seconds_bucket[5m])))

# Output tokens per second by model
sum by (gen_ai_request_model) (rate(gen_ai_client_token_usage_total{gen_ai_token_type="output"}[5m]))

# Cost in the last 24 hours (custom counter from Unit 2)
sum(increase(triage_model_cost_usd_total[24h]))

# Share of requests blocked by any guardrail
sum(rate(guardrail_decisions_total{decision="block"}[15m]))
  / sum(rate(triage_request_seconds_count[15m]))

# Triage requests by prompt version and category
sum by (prompt_version, category) (rate(triage_request_seconds_count[1h]))
```



A dashboard for an LLM feature

- Row 1, traffic and latency: requests per minute, p50 and p95 latency end to end and per model, errors and timeouts
- Row 2, cost: tokens by type and model, cost per hour and per day, cost per 1,000 requests, projection against budget
- Row 3, quality: validation failures and retries, refusal rate, incomplete responses, online evaluation score, feedback rate
- Row 4, safety: guardrail decisions by guardrail, PII and injection detections, escalations to humans
- Row 5, retrieval and tools: retrieval latency, empty retrievals, tool calls and tool errors
- Every panel can be filtered by prompt version and model, which is essential during rollouts



Alerts on cost, latency and quality

- Alert on symptoms that need action, tied to the service level objectives o
  - Cost: spend in the last hour projected over a day exceeds the daily budget
  - Latency: p95 above the objective for a sustained period
  - Quality: validation failure rate, refusal rate outside its band, online evaluation score below baseline
  - Safety: sudden increase in guardrail blocks or injection detections



Alerts on cost, latency and quality

```yaml
groups:
- name: triage-llm
  rules:
  - alert: TriageCostAboveBudget
    expr: sum(increase(triage_model_cost_usd_total[1h])) * 24 > 50
    for: 30m
    labels: { severity: warning }
    annotations:
      summary: "Projected daily LLM cost above $50"
      runbook: "https://wiki.example/runbooks/triage-cost"

  - alert: TriageLatencyHigh
    expr: histogram_quantile(0.95, sum by (le) (rate(gen_ai_client_operation_seconds_bucket[10m]))) > 4
    for: 15m
    labels: { severity: page }

  - alert: GuardrailBlockSpike
    expr: sum(rate(guardrail_decisions_total{decision="block"}[15m]))
          > 3 * sum(rate(guardrail_decisions_total{decision="block"}[1d] offset 1d))
    for: 10m
    labels: { severity: warning }
```



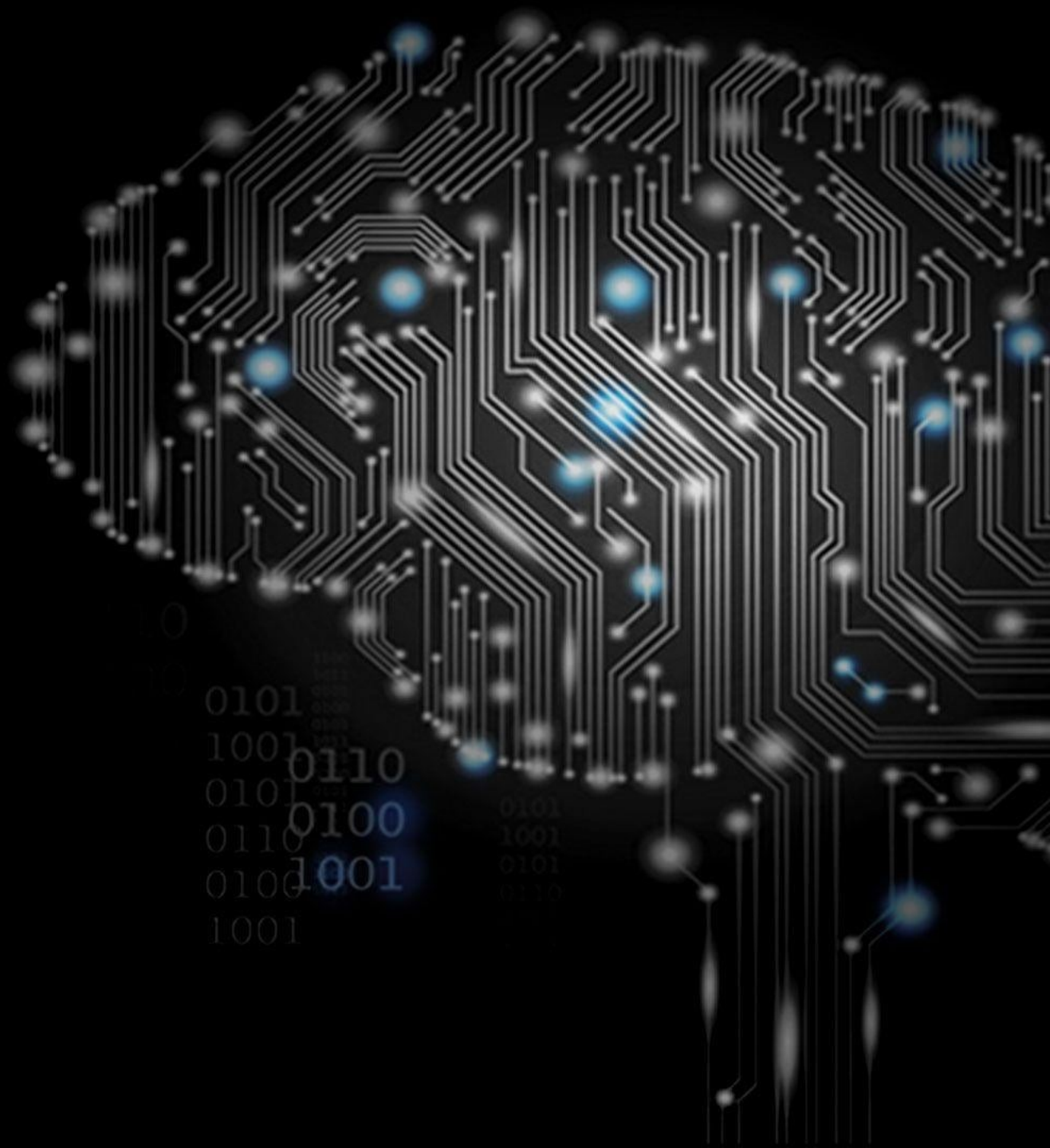
Alerts on cost, latency and quality

- Alert on symptoms that need action, tied to the service level objectives o
  - Cost: spend in the last hour projected over a day exceeds the daily budget
  - Latency: p95 above the objective for a sustained period
  - Quality: validation failure rate, refusal rate outside its band, online evaluation score below baseline
  - Safety: sudden increase in guardrail blocks or injection detections



:Designing alerts people will act on

- Every alert has an owner, a severity and a runbook describing the first steps
- Page only for problems that need immediate human action; send the rest to a ticket queue or channel
- Prefer rates and ratios over absolute counts, and require the condition to persist before firing
- Include the prompt version and model in alert context, so responders can check recent changes first
- Review alerts after every incident: remove the ones nobody acted on, add the ones that were missing



Observability and QA

4. Logging prompts and responses safely

Why log content, and why it is risky

- Reasons to keep prompt and response content
  - Debugging incorrect answers, investigating complaints, audit evidence, building golden datasets and online evaluation samples
- Risks
  - Customer messages contain personal and financial information
  - Prompts contain bank policy and internal instructions
  - Logs are often widely accessible, long-lived and copied to other systems
- The bank's privacy, records management and data residency rules apply to LLM content exactly as to any customer data



Spring AI content logging settings

- All content logging is off by default, and each property carries a documented warning about exposing sensitive data
- Use these settings in local development and controlled test environments only
- In production, record content through a dedicated, redacted, access-controlled path instead (next slides)

```yaml
spring:
  ai:
    chat:
      observations:
        log-prompt: false
        log-completion: false
      client:
        observations:
          log-prompt: false
          log-completion: false
    tools:
      observations:
        include-content: false        # tool arguments and results
    vectorstore:
      observations:
        log-query-response: false     # retrieved documents
```

Redaction before anything is written

- Redact at the point of capture, before content reaches logs, traces or analytics
  - Reuse the PII masker from Unit 5; the same code protects the model input and the logs
- Store masked text for routine use; store unmasked text only where policy requires it, in a restricted store
- Redact again in the telemetry pipeline (for example in the OpenTelemetry Collector) as a second layer
- Test redaction with the same PII examples used for guardrail tests

```
public record InteractionLog(
        Instant timestamp,
        String correlationId,
        String promptVersion,
        String model,
        String maskedInput,
        String maskedOutput,
        List<String> retrievedDocIds,
        List<String> guardrailDecisions,
        int inputTokens,
        int outputTokens,
        BigDecimal cost,
        long latencyMs,
        String outcome) {}

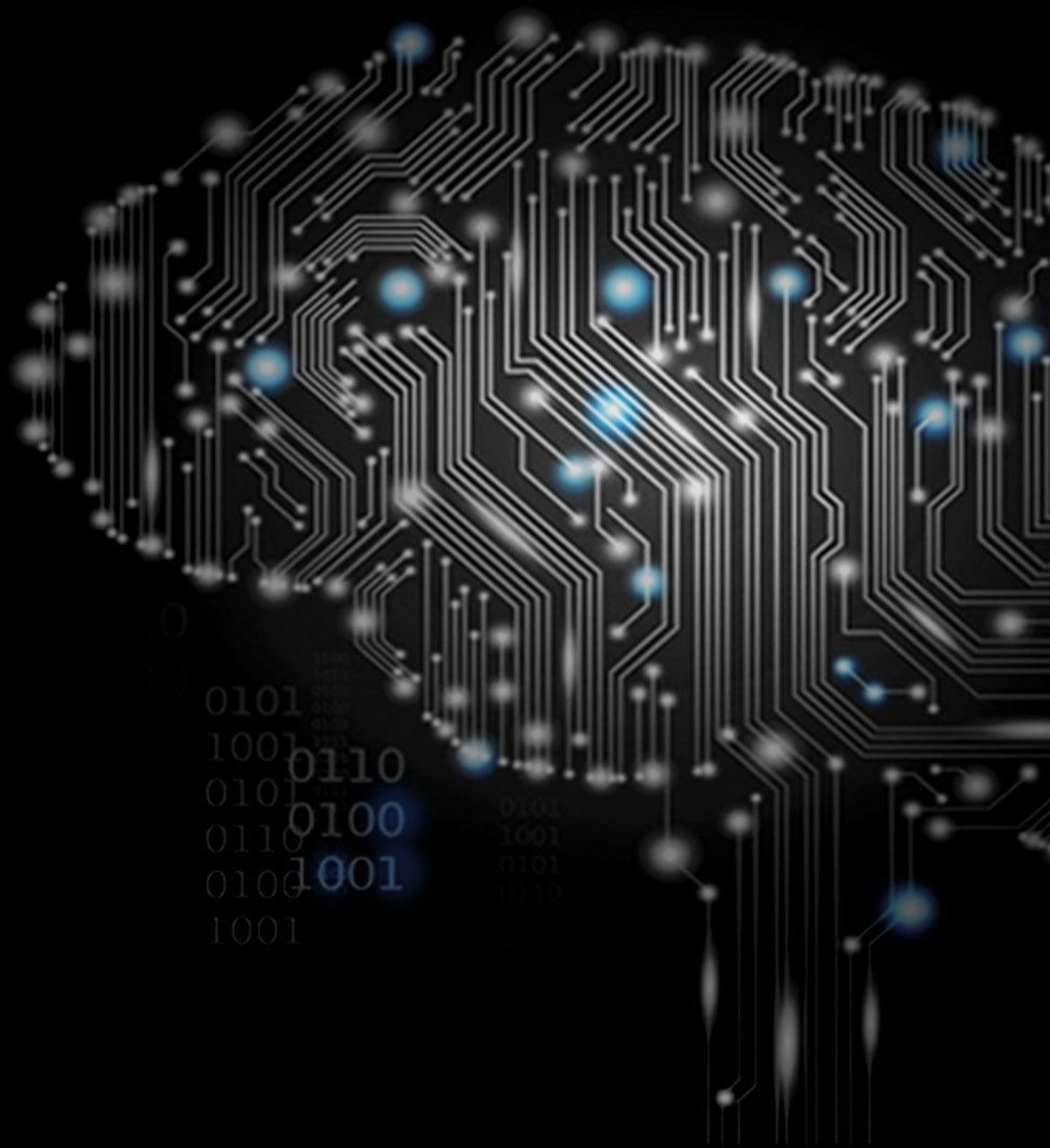
void record(TriageContext ctx, TriageResult result) {
    var entry = new InteractionLog(Instant.now(), ctx.correlationId(), ctx.promptVersion(), ctx.model(),
            piiMasker.mask(ctx.customerMessage()), piiMasker.mask(result.suggestedReply()),
            ctx.retrievedDocIds(), ctx.guardrailDecisions(),
            ctx.inputTokens(), ctx.outputTokens(), ctx.cost(), ctx.latencyMs(), "triaged");
    interactionStore.save(entry);   // dedicated, access-controlled store
}
```



Retention and access control

- Store interaction records separately from general application logs
  - A dedicated store with its own access controls and audit of who reads it
- Grant access by role and need: support investigators, model validators, auditors
- Apply retention periods set by the bank's records policy, and delete on schedule
- Keep telemetry lean: traces and metrics carry IDs and counts; content stays in the controlled store, referenced by ID
- Keep data in approved locations; telemetry back ends and logging services are subject to data residency rules too





Observability and QA

5. QA in production

Why offline evaluation is not enough

- The golden dataset represents yesterday's understanding of the traffic
- Real traffic brings new topics, phrasing, languages and attack patterns
- Providers update models, and retrieval content changes, without a code release
- Production QA continuously answers the question: is the feature still as good as when we released it?



Online evaluation of sampled traffic

- Score a sample of real interactions with the evaluators from Unit 4, asynchronously, off the request path
- Rule-based checks on all traffic where they are cheap; LLM judges on a sample
- Publish scores as metrics tagged with prompt version and model, so they appear on dashboards and alerts
- Route low-scoring interactions to a review queue for human assessment
- Respect privacy: evaluate masked content where possible, inside approved infrastructure



Online evaluation of sampled traffic

```
@Component
class OnlineEvaluator {

    private final ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();
    private final double sampleRate;

    void maybeEvaluate(InteractionLog entry, String question, List<Content> context) {
        if (ThreadLocalRandom.current().nextDouble() >= sampleRate) return;
        executor.submit(() -> {
            EvaluationResponse eval = relevancy.evaluate(
                    new EvaluationRequest(question, context, entry.maskedOutput()));
            meterRegistry.counter("online.eval",
                    "evaluator", "relevancy",
                    "result", eval.isPass() ? "pass" : "fail",
                    "prompt.version", entry.promptVersion()).increment();
            if (!eval.isPass()) reviewQueue.add(entry);
        });
    }
}
```



User feedback signals

- Explicit feedback
  - Thumbs up or down, or a short reason, from customers or staff
- Implicit feedback, often more reliable and plentiful
  - How much an agent edits a suggested reply before sending it
  - Whether a triaged message is re-routed to another team
  - Escalations, repeat contacts on the same issue, abandoned conversations
- Record feedback with the correlation ID, so it links back to the trace and interaction record

```
@PostMapping("/replies/{correlationId}/sent")
void replySent(@PathVariable String correlationId, @RequestBody SentReply sent) {
    InteractionLog original = interactionStore.find(correlationId);
    int distance = LevenshteinDistance.getDefaultInstance()
            .apply(original.maskedOutput(), piiMasker.mask(sent.text()));
    double editRatio = (double) distance / Math.max(1, original.maskedOutput().length());
    meterRegistry.summary("reply.edit.ratio", "prompt.version", original.promptVersion())
            .record(editRatio);
}
```

Drift detection

- Input drift: the mix of incoming requests changes
  - Category distribution, language mix, message length, new topics, injection attempt rate
- Output drift: the system's behaviour changes
  - Refusal rate, reply length, category distribution assigned, validation failures, online evaluation scores
- Model drift: the provider changes the model behind a name or alias
  - Track the response model reported in metadata, and pin versions where possible
- Detect drift by comparing a recent window with a reference window, per prompt version and model

```
# Share of FRAUD classifications today compared with the same day last week
  sum(increase(triage_request_seconds_count{category="FRAUD"}[1d]))
/ sum(increase(triage_request_seconds_count[1d]))
-
  sum(increase(triage_request_seconds_count{category="FRAUD"}[1d] offset 7d))
/ sum(increase(triage_request_seconds_count[1d] offset 7d))
```



Release gates

- A release passes through several gates, each with defined criteria
  - Offline gate: evaluation suite and guardrail tests pass in CI (Unit 4)
  - Pre-production gate: the new version runs on recorded or mirrored traffic, and scores match or exceed the current version
  - Production gate: a canary receives a small share of traffic, and live metrics stay within objectives before full rollout
- Gates apply to prompt changes, model changes, guardrail changes and retrieval changes alike
- Record gate results as part of the change record for model risk management



Release gates

- Route a small, stable share of traffic to the new version, compare, then increase or roll back
- Assign versions deterministically by hashing a stable key, so each customer sees one version consistently
- Tag all metrics and traces with the version, so dashboards compare canary and baseline side by side
- Roll back by configuration, immediately, if canary metrics breach objectives

```
@ConfigurationProperties("rollout.triage")
public record RolloutProperties(String baseline, String canary, int canaryPercent) {}

PromptRef selectVersion(String customerId) {
    int bucket = Math.floorMod(customerId.hashCode(), 100);
    String version = bucket < rollout.canaryPercent() ? rollout.canary() : rollout.baseline();
    return new PromptRef("triage", version);
}
```

```
rollout:
  triage:
    baseline: v2
    canary: v3
    canary-percent: 5
```

Canary rollouts of prompts and models

- Route a small, stable share of traffic to the new version, compare, then increase or roll back
- Assign versions deterministically by hashing a stable key, so each customer sees one version consistently
- Tag all metrics and traces with the version, so dashboards compare canary and baseline side by side
- Roll back by configuration, immediately, if canary metrics breach objectives
-

```java
@ConfigurationProperties("rollout.triage")
public record RolloutProperties(String baseline, String canary, int canaryPercent) {}

PromptRef selectVersion(String customerId) {
    int bucket = Math.floorMod(customerId.hashCode(), 100);
    String version = bucket < rollout.canaryPercent() ? rollout.canary() : rollout.baseline();
    return new PromptRef("triage", version);
}
```

```yaml
rollout:
  triage:
    baseline: v2
    canary: v3
    canary-percent: 5
```

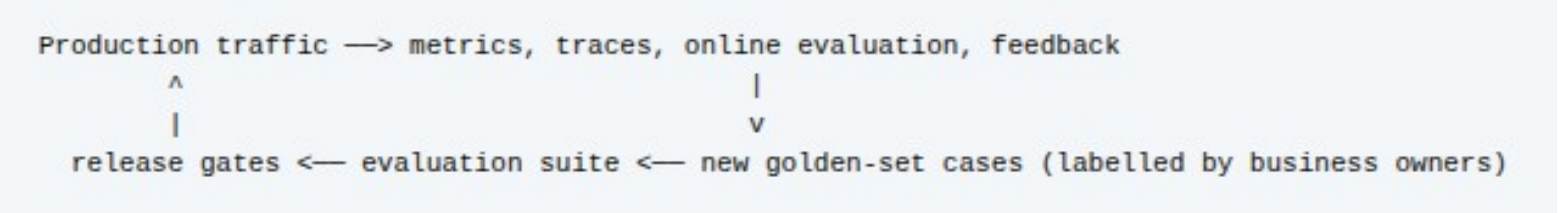
A/B tests of prompts

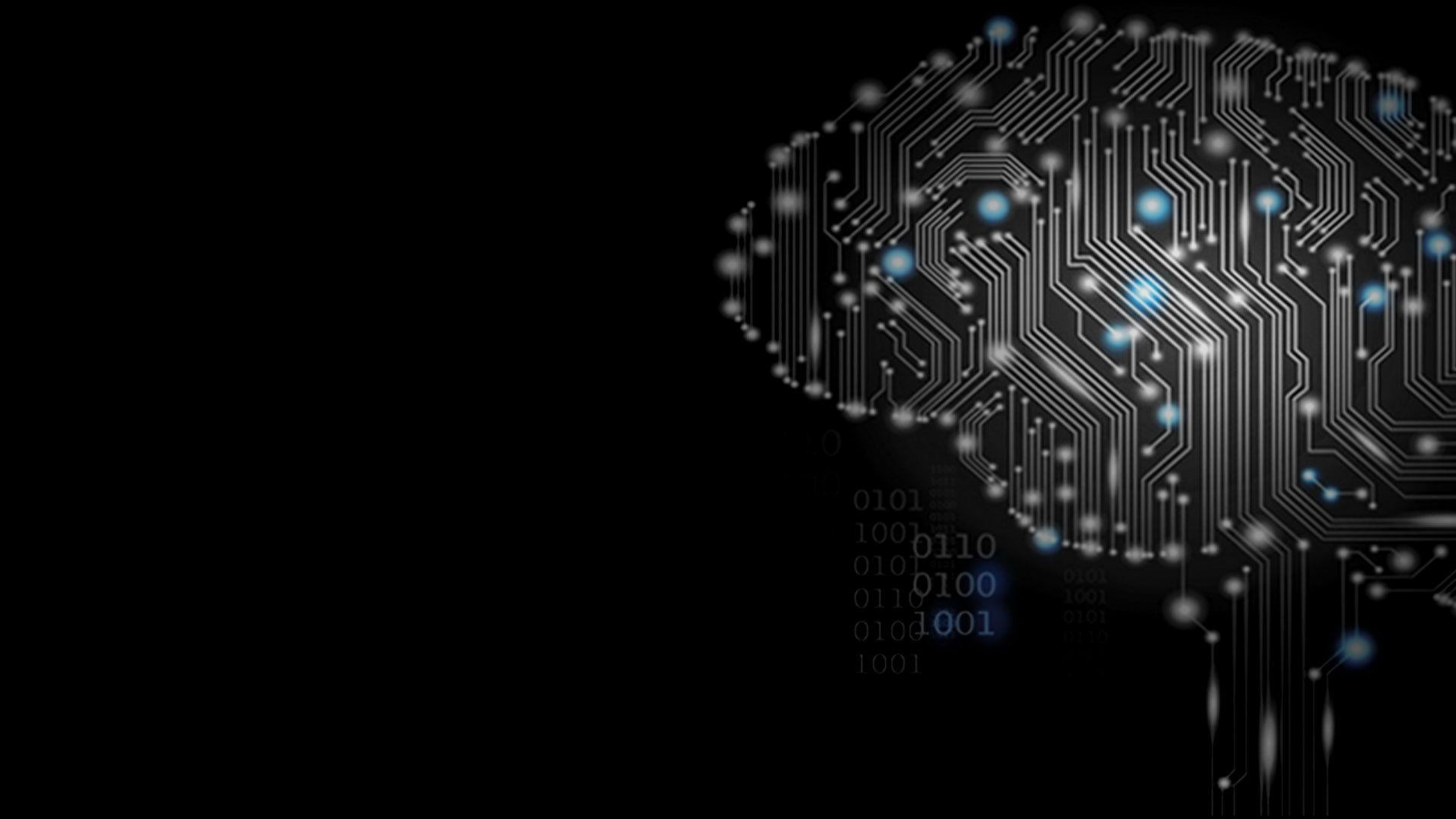
- A canary asks "is the new version safe?"; an A/B test asks "is the new version better?"
- Randomly assign traffic between two versions that have both passed all gates
- Choose the primary measure in advance: re-routing rate, edit ratio, online evaluation score, cost per request
- Run until the sample is large enough to detect the difference that matters; small differences need large samples
- Consider customer impact and fairness: both variants must meet all compliance requirements



Closing the loop

- Production signals feed development
  - Low online scores, negative feedback, re-routes and incidents become candidate golden-set cases
  - Business owners label them; they join the dataset through a reviewed change (Unit 4)
- The next prompt or model change is evaluated against the enlarged dataset before release
- Over time, the evaluation suite reflects real traffic more closely, and the same failure does not recur




0101
1001
0110
0100
1001